

محرك التجارة الإلكترونية

عالي الأداء

مادة: البرمجة المتوازية

الأستاذ: حذيفة عقيل

الطالب: وسام محمد سليم العبد

الطالب: محمد أديب بشار صباهي

الطالب: عبد الرحمن عبدالله بهلوان

الطالب: رنيم محمود قنوت

التاريخ: 18 مايو 2026

1. ملخص المشروع

تطبيق (Backend (Laravel 13 + Octane/Swoole) لمتجر إلكتروني يتحمل طلبات متزامنة كثيرة. يركز على: منع أخطاء المخزون (Race Condition)، إدارة السعة، الطوابير للمهام الثقيلة، وقياس الأداء تحت 100 مستخدم.

التشغيل: composer octane + php artisan queue:work — حساب تجريبي:

demo@highperformance.test / password

ملاحظة للتسليم: استخدمنا أداة الاختبار — php artisan benchmark:stress بدلاً من JMeter.

2. المتطلبات العشرة — جدول التنفيذ

#	المتطلب	التنفيذ في المشروع
1	سلامة بيانات متزامنة	InventoryService + lockForUpdate ()
2	إدارة موارد السيرفر	CapacityControlMiddleware
3	معالجة غير متزامنة (طوابير)	GenerateOrderInvoiceJob, SendOrderNotificationJob

ProcessDailySalesBatchJob + sales:process-daily	Batch processing	4
CircuitBreakerMiddleware + طوابير متعددة	توزيع الحمل	5
ProductCacheService (store: database)	Cache للقراءات	6
version + Pessimistic lock عمود	Concurrency control	7
Transaction: مخزون + طلب + رصيد + دفع محاكى	ACID	8
benchmark:stress --users=100 (القسم 5)	Stress test 100 مستخدم	9
benchmark:bottlenecks + /metrics	Benchmark / مراقبة	10

3. المعمارية

```

Client (Postman / stress tool)
  |
  v
+-----+
| Laravel Octane (Swoole) |
| Middleware: capacity + |
|           circuit breaker|
+-----+
  |
  +-----+-----+
  v             v
Actions         /metrics (Prometheus)
  |
Services: Inventory (lock), Payment, ProductCache
  |
MariaDB/SQLite      Queue workers (invoices, notifications)

```

4. Race Condition — قبل وبعد القفل

الأمر: `php artisan concurrency:race-demo 1 --attempts=30`

يبدأ المخزون من 10، ويُنفَّذ 30 خصم متوازي. بدون قفل يفقد التحديثات؛ مع `lockForUpdate` النتيجة صحيحة.

4.1 بدون قفل (UNSAFE)

```
Stock reset to 10. Running 30 PARALLEL unsafe decrements..  
UNSAFE final stock: 5 (expected 0)
```

النتيجة: 5 بدل 0 — فقدان تحديثات (lost updates).

4.2 مع `lockForUpdate` (SAFE)

```
Stock reset. Running 30 PARALLEL safe decrements (lockForUpdate)..  
SAFE final stock: 0
```

النتيجة: 0 — كل الخصومات طُبِّقَتْ بشكل صحيح.

5. اختبار الضغط — 100 مستخدم متزامن

الأمر (مع Octane شغال):

```
php artisan benchmark:stress --users=100
```

المؤشر	القيمة
عدد المستخدمين المحاكين	100
إجمالي الطلبات HTTP	134
مدة التنفيذ (ثانية)	2.202
طلبات في الثانية (RPS)	60.86
طلبات ناجحة	134
طلبات فاشلة	0
رموز HTTP	134 :200

609.27	زمن الاستجابة p50 (ms)
2133.68	زمن الاستجابة p95 (ms)
2181.24	أقصى زمن (ms)

المزيج: تسجيل دخول + GET /api/wallet + GET /api/product — يحاكي مسار مستخدم حقيقي.

```
+-----+-----+-----+-----+-----+-----+-----+
---+
| users | total_requests | duration_seconds | rps    | success | failed | status_codes |
| 100   | 134             | 2.202           | 60.86  | 134     | 0      | {"200":134}  |
+-----+-----+-----+-----+-----+-----+-----+
---+
```

6. واجهة API (Postman) — نماذج الاستجابة

Collection: postman/HighPerformance-User-APIs.postman_collection.json

6.1 تسجيل الدخول — POST /api/auth/login

```
{
  "status": "success",
  "data": {
    "user": {
      "name": "Demo User",
      "email": "demo@highperformance.test",
      "balance": "10000.00"
    },
    "token": "18|..."
  }
}
```

6.2 الرصيد — GET /api/wallet

```
{
  "status": "success",
  "data": {
    "balance": "10000.00",
    "version": 1
  }
}
```

6.3 المنتجات — GET /api/product

```
{
  "status": "success",
  "current_page": 1,
  "data": [
    { "id": 1, "name": "سماعات لاسلكية", "price": "89.99", "stock": 48 },
    { "id": 2, "name": "لوحة مفاتيح ميكانيكية", "price": "129.00", "stock": 100 }
  ]
}
```

6.4 سلة + طلب مدفوع — POST /api/cart ثم POST /api/order

```
{
  "status": "success",
  "message": "Order placed with simulated payment.",
  "data": {
    "id": 3,
    "order_status": "confirmed",
  }
}
```

```

    "payment_status": "paid",
    "payment_reference": "PAY-KGHNGWVIUHLV",
    "total": "179.98"
  }
}

```

الدفع محاكي داخل Transaction (ACID): خصم مخزون + خصم رصيد + تأكيد الطلب.

6.5 فائورة غير متزامنة (Queue)

```

storage/app/invoices/INV-00000001.json
{
  "order_id": 1,
  "total": "179.98",
  "lines": [{ "product": "سماعات لاسلكية", "qty": 2, "price": "179.98" }]
}

```

7. Benchmark — نقاط الاختناق

الأمر: `php artisan benchmark:bottlenecks`

النتيجة: لا اختناق حرج في العينة الحالية. توصيات النظام:

- تشغيل Octane + Swoole workers لزيادة throughput
- تشغيل queue workers منفصلين لكل طابور
- كاش المنتجات على database (ليس octane table للقوائم الكبيرة)

8. AOP — مراقبة التزامن

ConcurrencyAspect يسجل مدة عمليات: قفل المخزون، الدفع المحاكى، وطلبات HTTP — بدون تكرار كود القياس في كل Action.

9. ماذا تعلمت

- Race Condition لا يظهر في طلب واحد — يظهر فقط تحت طلبات متوازية؛ الحل: قفل على مستوى الصف.
- Octane يحتفظ بالحالة بين الطلبات — لا نخزن كائنات Eloquent في كاش Octane الصغير.
- فصل الفواتير والإشعارات في Queue يحسن زمن استجابة إنشاء الطلب.
- اختبار 100 مستخدم يمكن تنفيذه بأمر Artisan وليس بالضرورة JMeter.

10. هيكل الملفات المسلمة

```
high_performance/  
  app/Services/Concurrency/InventoryService.php  
  app/Services/Payment/SimulatedPaymentService.php  
  app/Jobs/  
  postman/  
  submission/pdf/takrir-kamil.pdf ← هذا الملف بعد الطباعة  
  submission/sources/outputs/ ← مخرجات الأوامر
```

— نهاية التقرير —